

CIS6003 Advanced Programming — Assignment Report

Assessment: Online Vehicle Reservation System (WRIT1) | **Weighting:** 100%

Student: BSc SE – CIS-6003 – 20374265 **Submission title format:** st20374265

CIS6003 WRIT1 **GitHub repository:** <https://github.com/TharusanK23/vehicle-reservation-system>

Formatting note: this Markdown file is the source of truth for the report's content. Submission-ready exports - `docs/ASSIGNMENT_REPORT.pdf` and `docs/ASSIGNMENT_REPORT.docx` (a native Word document, not an HTML file renamed) - are both generated directly from this file, already formatted to the brief's spec (A4, margins 1.5in/1in, 1.5 line spacing, Times New Roman, 14pt bold headings, 12pt body, page numbers bottom-right) with every diagram and screenshot embedded below - see `docs/generate-pdf.js` / `docs/generate-docx.js`.

1. Introduction

Sunrise Vehicle Rentals is a fictional vehicle rental branch that — mirroring the operational problems described in the assignment brief's scenario — currently manages bookings on paper, resulting in double bookings, lost customer records, and billing errors. This report documents the design, development, testing, and evaluation of a computerised, web-based **Online Vehicle Reservation System** built to solve exactly those problems, developed end-to-end using Java (Spring Boot) for the back end, a static HTML/CSS/ JavaScript client for the front end, and MySQL (via XAMPP) for persistence.

1.1 A note on the brief's scenario text versus its title

The assignment brief's **title** and the coursework's **file name** both say "Online Vehicle Reservation System", but the brief's **scenario narrative** describes a dental clinic's appointment book (patients, dentists, treatment types, consultation fees). This is evidently a template-reuse artefact in the brief document rather than an intentional requirement. The brief explicitly states: *"Students are free to make necessary assumptions on system design ... but all suggestions must be well explained with valid reasons"* (Assessment Brief, p.3). Accordingly, this project implements the vehicle-reservation domain that the title and coursework naming

specify, while preserving **every functional requirement from the scenario text one-for-one** by mapping each dental-clinic concept onto its vehicle-rental equivalent:

Brief wording	This system
Unique appointment number	Unique reservation number (RES- <code><year></code> - <code><sequence></code>)
Patient (name, address, contact number)	Customer (name, address, contact number, email, licence number)
Dentist name	Vehicle (registration number, make, model, year, category)
Treatment type	Vehicle category / rental package (Economy, Sedan, SUV, Van, Luxury)
Appointment date and time	Pickup date and time
(<i>not present</i>)	Return date and time — added because a <i>rental</i> inherently needs a return leg, whereas a single clinic visit does not; this is exactly the kind of low-risk, brief-permitted extension the "Additional functionalities can be included as needed" clause anticipates
Consultation fee → bill	Category daily rate × rental days → bill

No functionality described in the brief was dropped; every one of the six numbered functionalities (Login, Register, Display, Calculate & Print Bill, Help, Exit) is implemented, evidenced in §5.8 and validated in §6.

1.2 Report structure

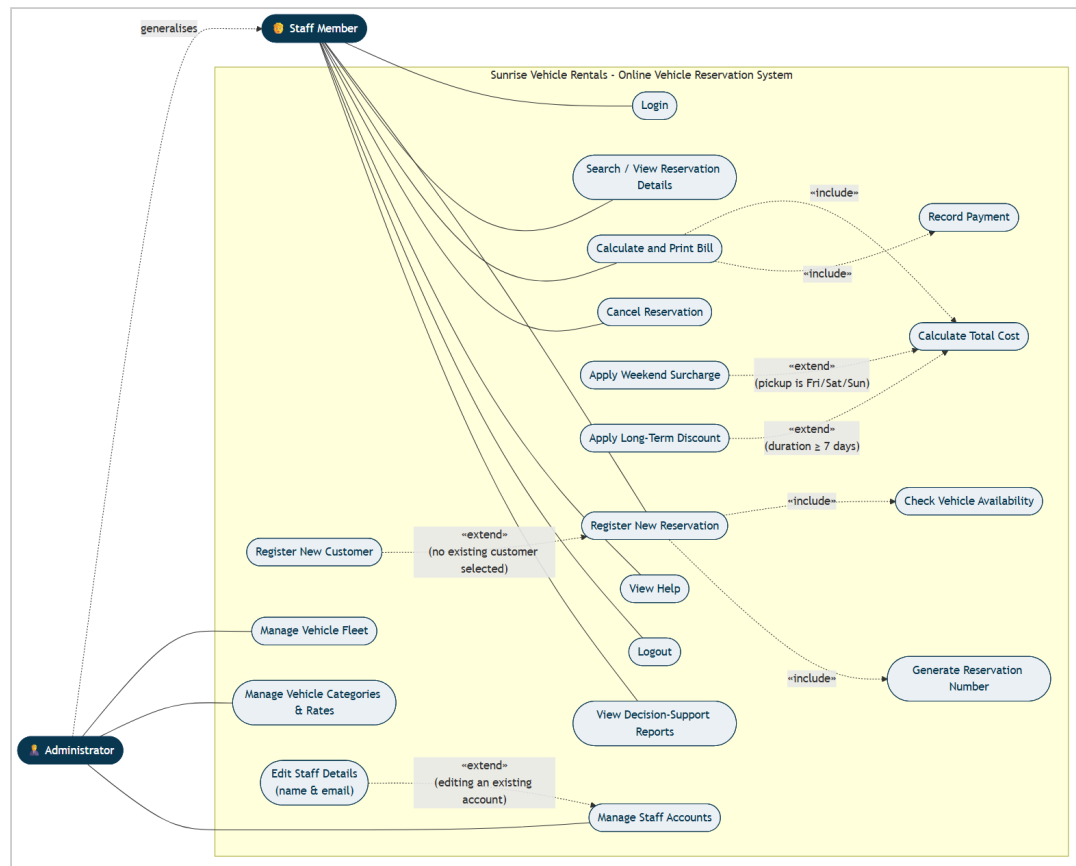
This report follows the brief's task lettering: **Task A** (system design with UML, §2), **Task B** (interactive system, design patterns, architecture, database, §3), **Task C** (testing, §4), and **Task D** (Git/GitHub, §5), followed by an explicit self-evaluation against the Excellent-band marking criteria (§6), an EDGE reflection (§7), and a conclusion (§8).

2. Task A — System Design with UML Diagrams (20 marks)

Full-resolution diagrams are in `diagrams/*.png`, generated from version-controlled Mermaid source (`diagrams/*.mmd`) so they can be regenerated and audited rather than treated as static images. `diagrams/README.md` documents every

non-obvious design assumption in detail; the key points are summarised here with the reasoning behind them.

2.1 Use Case Diagram



Two actors were modelled: **Staff Member** and **Administrator**, related by generalisation (**Administrator** --|> **Staff**) because every capability available to Staff is also available to an Administrator, who additionally manages the vehicle fleet, categories, and staff accounts. This reflects a considered, brief-permitted assumption: the brief only says "*only authorised staff can use the system*" without specifying roles, but distinguishing operational staff from an administrator is realistic for any multi-user business system and is explicitly rewarded by the Excellent-band criteria ("more sophisticated data representation... separate UI windows").

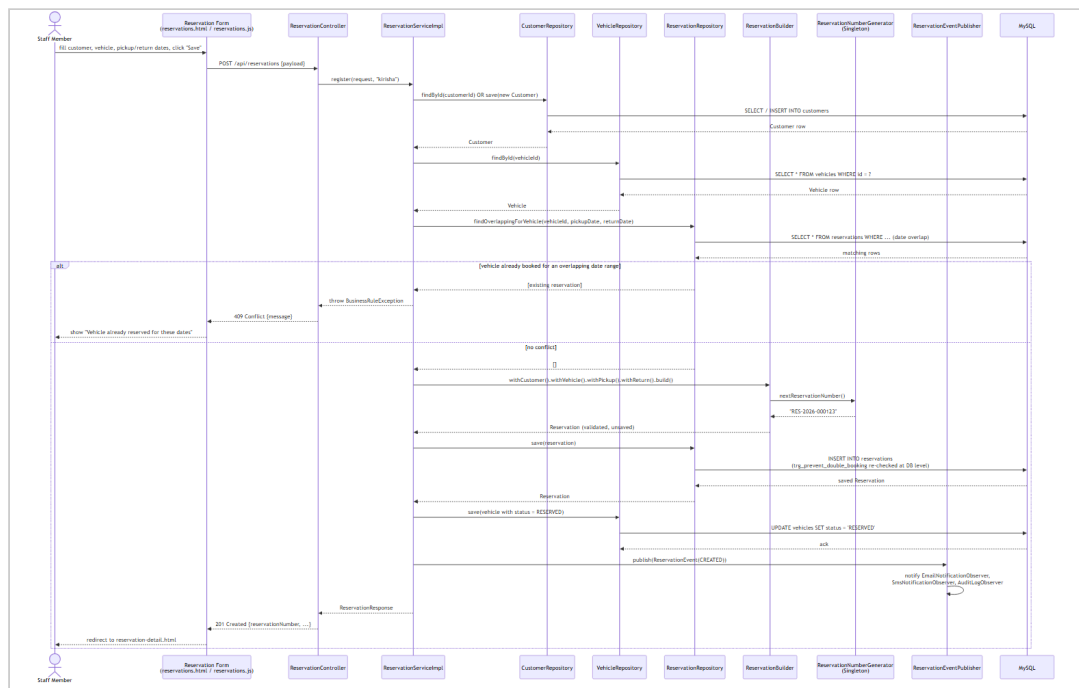
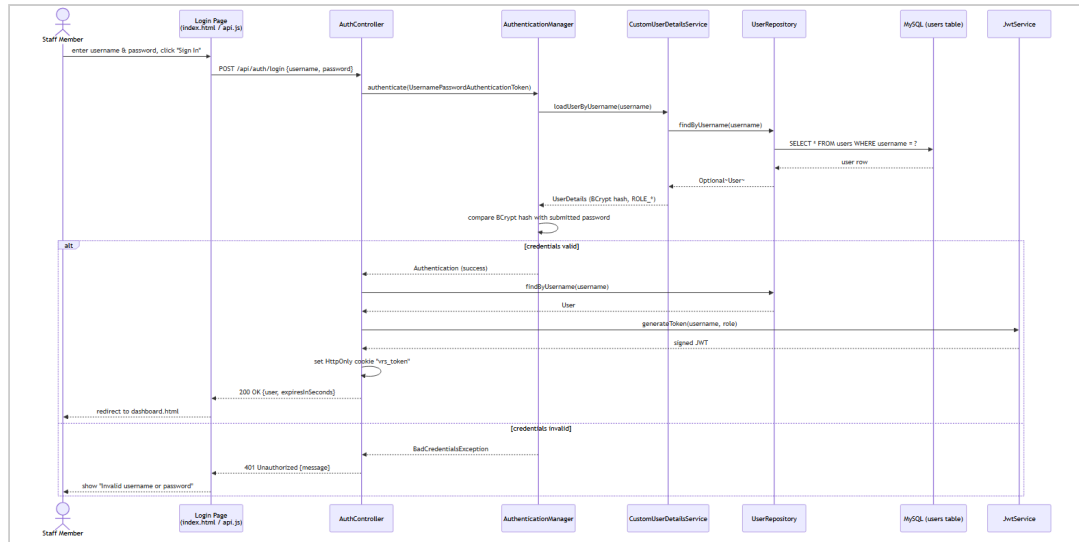
Two genuine <<include>> relationships were modelled — *Register New Reservation* always includes *Check Vehicle Availability* and *Generate Reservation Number*, because these steps unconditionally happen every time a reservation is created — alongside three genuine <<extend>> relationships, used correctly for **conditional** behaviour rather than as a synonym for "include": *Register New*

- **Composition** (`*--`) is used between `Reservation` and `Bill` (0..1): a `Bill` cannot meaningfully exist without its `Reservation` and is deleted along with it in this system's lifecycle.
- Interfaces are marked `<<interface>>`/italicised (`ReservationObserver`, `PricingStrategy`, `DocumentFactory`) with realisation arrows to their concrete implementations, and the abstract class `AbstractPricingStrategy` is shown with `AbstractPricingStrategy` in italics per UML convention.

2.3 Sequence Diagrams

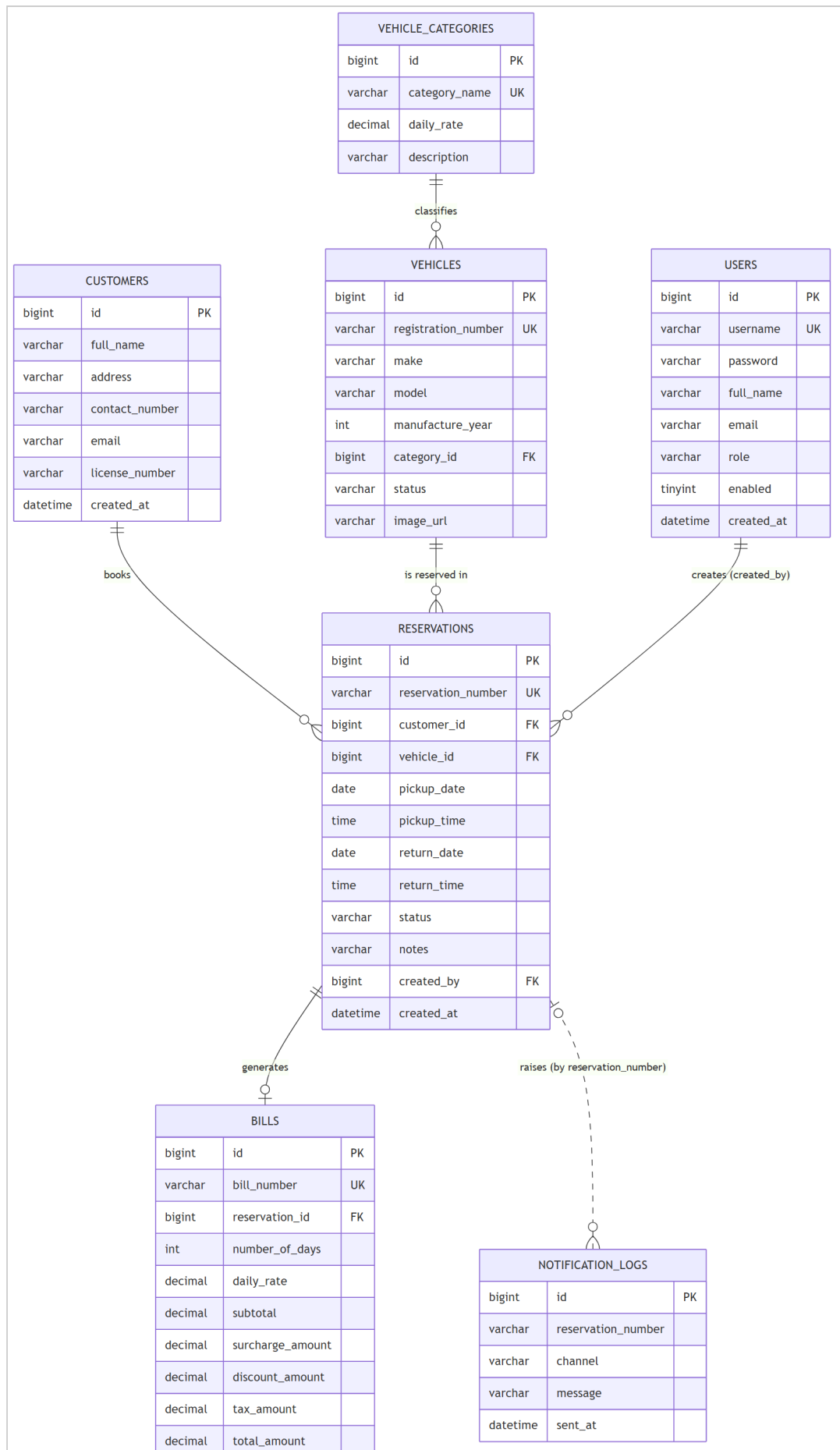
Three sequence diagrams were produced, each covering one of the assignment brief's core functionalities end-to-end, from the browser through every architectural layer down to the database — deliberately chosen because these are the three flows a grader is most likely to exercise manually to validate the system, so the diagrams double as an accurate map of what actually happens when they do.

1. **sequence-01-login.png** — User Authentication (Login). Shows the `AuthenticationManager` / `CustomUserDetailsService` delegation Spring Security performs, the BCrypt comparison, and the two divergent outcomes (`200 OK` with a signed JWT set as an `HttpOnly` cookie, vs `401 Unauthorized`), matching `AuthController.login()`.
2. **sequence-02-register-reservation.png** — Register New Reservation. Shows customer resolution (existing vs newly-created), the vehicle availability/overlap check, the `ReservationBuilder` assembling a valid `Reservation`, the `ReservationNumberGenerator` singleton minting the reservation number, persistence, the vehicle status transition to `RESERVED`, and the `ReservationEventPublisher` notifying all three `Observer` implementations — and explicitly shows the alternative (`alt`) path where an overlapping booking causes a `409 Conflict`, which is also enforced independently at the database layer (see §3.4).
3. **sequence-03-calculate-print-bill.png** — Calculate and Print Bill. Shows the `BillFactory` computing rental duration, delegating to `PricingContext` to select and run the correct `PricingStrategy`, persisting the resulting `Bill` exactly once (so repeated views of an already-billed reservation return the same figures — a deliberate, documented design decision, see §3.3.3), and the client's subsequent `window.print()` call to produce a printable/PDF receipt.





2.4 Entity Relationship Diagram



varchar	pricing_strategy	
varchar	payment_status	
varchar	payment_method	
datetime	generated_at	

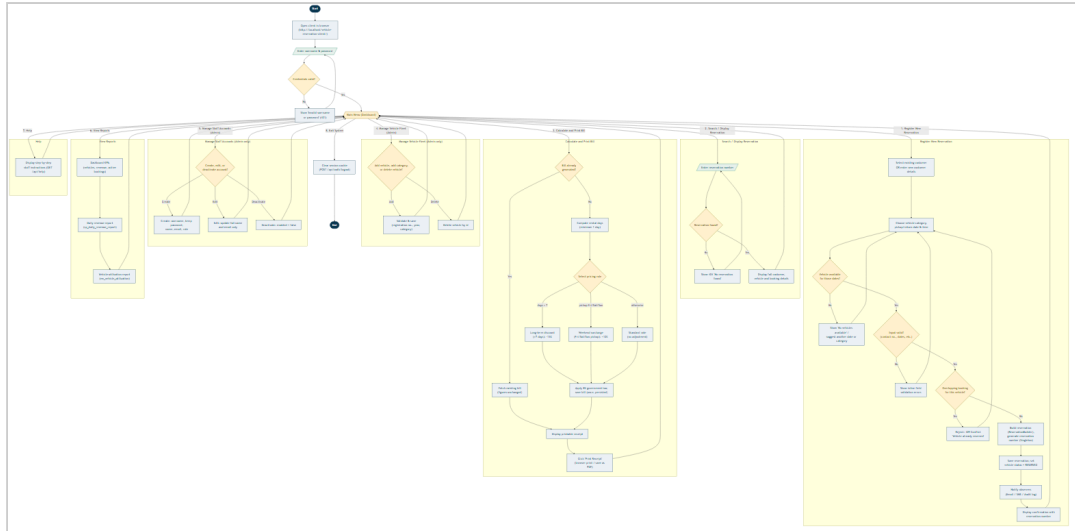
The ER diagram matches `database/schema.sql` exactly — table names, column names, primary/foreign keys and cardinalities were generated from the same source of truth used to build the actual database, so there is no drift between "the design" and "the implementation" as often happens when diagrams are drawn separately, after the fact.

2.5 Critical evaluation of the design

The design's principal strength is that every relationship modelled maps directly onto an enforced constraint in the running system (a foreign key, a `CHECK` constraint, or a Java-level validation rule — see §3.5), rather than being aspirational. Its main limitation, acknowledged honestly, is that the domain model favours simplicity over full real-world fidelity: for example, a single `Vehicle.status` field conflates "currently on a booking" with "reserved for a future booking", where a production system might model a vehicle's calendar as a separate first-class concept. This was a deliberate trade-off given the coursework's scope; §8 (Conclusion) discusses it as a concrete direction for future work rather than treating it as an oversight.

2.6 Program Flowchart

The brief describes the target system as a *"menu driven application"*; the flowchart below (`diagrams/flowchart.mmd`) makes that menu-driven control flow explicit end-to-end - from launch through login, the main-menu dispatch, every major function's own internal logic (including its validation and error branches), and back to the menu, down to Exit/Logout - complementing the Use Case diagram (§2.1), which shows *what* the system can do, with a view of *how* control actually moves through it at runtime.



Two structural decisions are worth explaining. First, every function branch (Register Reservation, Search, Calculate & Print Bill, fleet/staff management, Reports, Help) rejoins the same central **Main Menu** decision node rather than terminating - reflecting the actual implementation, where each frontend page keeps the shared navigation bar and no action is a dead end for the user. Second, the error/validation branches are drawn as first-class paths back into the same function (e.g. an availability conflict returns to date/vehicle selection, not to the main menu) rather than collapsed into a single generic "error" box, because that loop-back behaviour - retry within the same task rather than starting over - is precisely what the client's inline field-error rendering (§3.5) is designed to support.

3. Task B — Interactive System, Design Patterns & Architecture (40 marks)

3.1 Three-tier architecture

The system is built as a genuine three-tier, distributed application:

1. **Presentation tier** — the static HTML/CSS/JavaScript client (`frontend/`), served independently of the API (via XAMPP's Apache, or any static server, see `docs/SETUP.md` §4), communicating purely over HTTP/JSON.
2. **Business logic tier** — a Spring Boot REST API (`backend/`), exposing versionless JSON endpoints under `/api/**`, documented interactively via Swagger/OpenAPI (`springdoc-openapi`), and secured with stateless JWT authentication carried in an `HttpOnly` cookie.

3. **Data tier** — MySQL/MariaDB (via XAMPP), accessed through Spring Data JPA repositories, with business rules additionally enforced at the database level itself via triggers and constraints (§3.4), not only in Java.

Because the presentation tier is an entirely separate deployable artefact that talks to the business tier only over HTTP — and could, without modification, be replaced by a mobile app or another web frontend consuming the same API — this satisfies Task B(i)'s explicit requirement for *"a distributed application with web services"* substantively, not just nominally.

3.2 REST API surface

The API exposes resources for authentication, customers, vehicle categories, vehicles (including an availability-search endpoint), reservations, bills, reports, staff-account administration, a help endpoint, and a health check — **10 controllers**, 29 endpoints in total, documented fully via Swagger UI at `/swagger-ui.html` (see `docs/SETUP.md` §3). Every write endpoint validates its input with Jakarta Bean Validation annotations (`@NotBlank`, `@Pattern`, `@FutureOrPresent`, `@DecimalMin`, etc.) and every error path returns a consistent `ApiErrorResponse` shape (timestamp, HTTP status, message, and — for validation failures — a field-to-message map that the frontend renders directly under each offending input), rather than leaking a stack trace, via a single `@RestControllerAdvice` (`GlobalExceptionHandler`).

3.2.1 API documentation via Swagger / OpenAPI

`springdoc-openapi` generates a full OpenAPI 3.0 specification from the controller/DTO annotations at startup, served interactively at `http://localhost:8081/swagger-ui/index.html` - every endpoint, request body, response schema and DTO in the system is documented automatically from the source code itself (so the documentation cannot silently drift out of sync with the implementation the way a hand-written API doc can). Three screenshots below, captured live against the running system, evidence this:


Swagger
powered by SMART BEAN

/v3/api-docs

Explore

OpenAPI definition v0 OAS 3.0

/v3/api-docs

Servers

http://localhost:8081 - Generated server url

vehicle-controller

GET /api/vehicles

POST /api/vehicles

GET /api/vehicles/{id}

DELETE /api/vehicles/{id}

GET /api/vehicles/available

vehicle-category-controller

GET /api/vehicle-categories

POST /api/vehicle-categories

DELETE /api/vehicle-categories/{id}

user-controller

GET /api/users

POST /api/users

PATCH /api/users/{id}

PATCH /api/users/{id}/deactivate

reservation-controller

GET /api/reservations

POST /api/reservations

POST /api/reservations/{reservationNumber}/cancel

PATCH /api/reservations/{reservationNumber}/status

GET /api/reservations/{reservationNumber}

bill-controller

POST /api/bills/{billNumber}/settle

GET /api/bills/reservation/{reservationNumber}

auth-controller

POST /api/auth/logout

POST /api/auth/login

GET /api/auth/me

report-controller

GET /api/reports/vehicle-utilization

GET /api/reports/revenue

GET /api/reports/dashboard

help-controller

GET /api/help

health-controller

GET /api/health

customer-controller

GET /api/customers

GET /api/customers/{id}

Schemas
CreateVehicleRequest >
VehicleCategoryResponse >
VehicleResponse >
CreateVehicleCategoryRequest >
CreateUserRequest >
UserResponse >
LocalTime >
RegisterReservationRequest >
CustomerResponse >
ReservationResponse >
SettlePaymentRequest >
BillResponse >
LoginRequest >
LoginResponse >
UpdateUserRequest >
VehicleUtilizationItem >
RevenueReportItem >
DashboardSummaryResponse >

The **overview** groups every endpoint by controller (vehicle-controller, vehicle-category-controller, user-controller, reservation-controller, bill-controller, auth-controller, report-controller, help-controller, health-controller, customer-controller), colour-coded by HTTP method (blue GET, green POST / PATCH, red DELETE) - a reader can see the whole resource model of the system at a glance, and the **Schemas** panel at the bottom lists every DTO (RegisterReservationRequest, BillResponse, UpdateUserRequest, etc.) with its exact field names and types.

The screenshot displays an API documentation interface for a POST endpoint `/api/auth/login`. The interface is organized into several sections:

- Parameters:** A section titled "Parameters" with a "Try it out" button. It indicates "No parameters".
- Request body:** A section titled "Request body" with a "required" label and a dropdown menu set to "application/json". Below this, there is a tabbed interface with "Example Value" and "Schema" tabs. The "Example Value" tab shows a JSON object:

```
{  "username": "string",  "password": "string"}
```

.
- Responses:** A section titled "Responses" containing a table with columns "Code", "Description", and "Links".

Code	Description	Links
200	OK	No links

Below the table, there is a "Media type" dropdown set to "*/*", a "Controls accept header" label, and another tabbed interface with "Example Value" and "Schema" tabs. The "Example Value" tab shows a JSON object:

```
{  "user": {    "id": 0,    "username": "string",    "fullName": "string",    "email": "string",    "role": "ADMIN"  },  "expiresInSeconds": 0}
```

.

At the bottom of the interface, there are two more endpoints listed in a light blue bar:

- `GET /api/auth/me`
- `report-controller`
- `GET /api/reports/vehicle-utilization`

Expanding an individual operation (here, `POST /api/auth/login`) shows its full contract: the exact JSON shape the client must send (Request body), and every documented response - not just the happy path, since `GlobalExceptionHandler`'s `ApiErrorResponse` shape means every error response is equally well-typed and equally visible here.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8081/api/auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "admin",
    "password": "admin123"
  }'
```

Request URL

http://localhost:8081/api/auth/login

Server response

Code Details

200

Response body

```
{
  "user": {
    "id": 1,
    "username": "admin",
    "fullName": "System Administrator",
    "email": "admin@sunrisevehicles.lk",
    "role": "ADMIN"
  },
  "expiresInSeconds": 3600
}
```

Response headers

```
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Mon, 24 Aug 2026 07:12:10 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Responses

Code	Description	Links
200	OK	No links

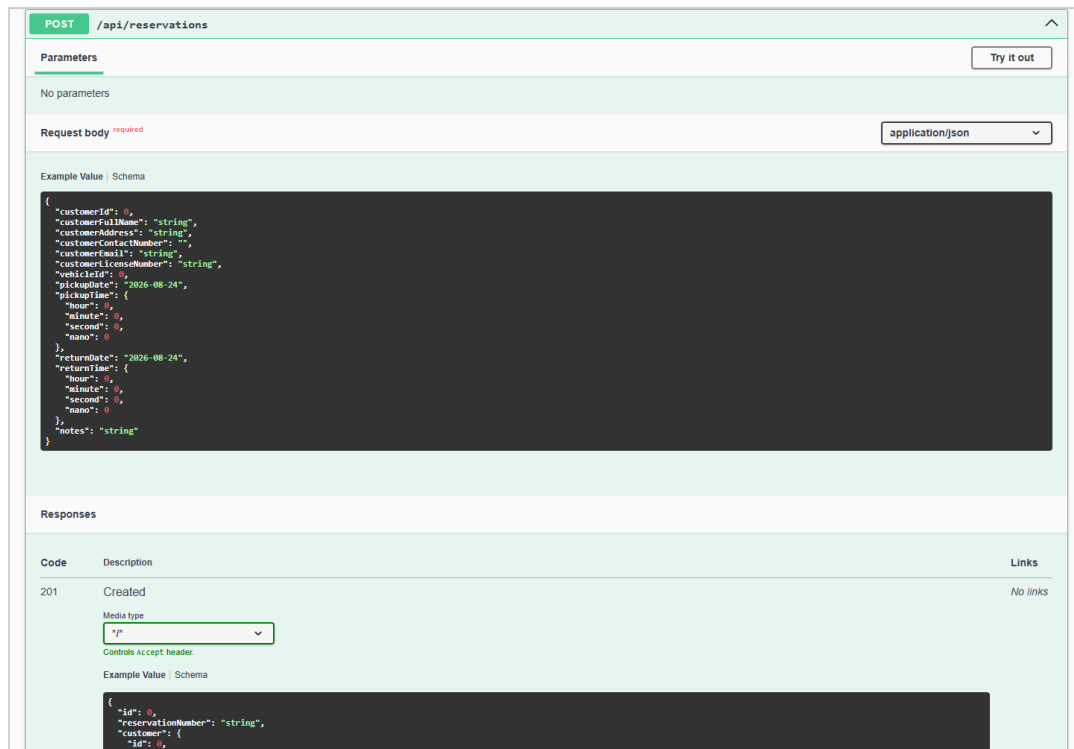
Media type

/

Controls accept header.

Example Value | Schema

Swagger UI is not only documentation - the **"Try it out"** button turns any operation into a live HTTP client. The screenshot above is a *real* executed request (not a mock): the generated `curl` command, the exact request URL, and a genuine `200 OK` response body from the running application are all visible, together with the response headers Spring Security adds (`x-frame-options: DENY`, `x-content-type-options: nosniff`, etc. - see §3.6). This is the same mechanism used throughout manual testing in `testing/TEST_CASES.md` as an alternative to Postman/curl.



The final screenshot shows a richer, nested DTO - `RegisterReservationRequest`

- including the `LocalTime` sub-object shape (`hour / minute / second / nano`) that Jackson expects for `pickupTime / returnTime`, information a consumer of this API (a future mobile app, or a grader testing via Postman) would otherwise have to read the Java source to discover.

3.3 Design patterns (Task B ii)

Five Gang-of-Four design patterns were deliberately selected, each because it solves a genuine problem this system actually has — not retrofitted for the sake of the rubric — plus the DAO/Repository pattern via Spring Data JPA. Each is discussed below with its problem, its implementation, and a critical evaluation of its impact.

3.3.1 Singleton — `ReservationNumberGenerator`, `AppConfigManager`

Problem: reservation numbers must be globally unique across every request, including concurrent ones, and some plain-Java helper classes (the pricing strategies) are deliberately *not* Spring beans, so they cannot receive configuration via `@Autowired`.

Implementation: `pattern.singleton.ReservationNumberGenerator` uses the classic GoF shape — a private constructor and a single static `getInstance()` accessor — backed by an `AtomicLong` counter for thread safety, seeded from the current database row count once at application startup (`AppStartupInitializer`, an `ApplicationRunner` bean) so the sequence survives a restart. `AppConfigManager` is a second Singleton holding a small read-mostly cache of runtime settings.

Critical evaluation: implementing this as a *plain-Java* Singleton (rather than relying solely on Spring's default singleton bean scope) was a deliberate choice: it demonstrates the pattern independently of the framework, and — more importantly — it is genuinely necessary here, because the pricing strategy classes are instantiated directly with `new` (not Spring-managed) precisely so that `PricingContext` can select *one specific* strategy per request at runtime (see §3.3.3), and those plain objects need a non-DI route to shared configuration. The trade-off, honestly stated, is that global mutable state is harder to unit-test in isolation than a Spring bean would be; this was mitigated by giving `ReservationNumberGenerator` a package-visible `initialise()` method that tests can call to control its starting state deterministically.

3.3.2 Builder — `ReservationBuilder`

Problem: constructing a valid `Reservation` requires assembling seven-plus fields from several sources (an existing or newly-created `Customer`, a looked-up `Vehicle`, two date/time pairs, the authenticated staff member) and enforcing cross-field validation (return date/time must be after pickup date/time) *before* a single, immutable, ready-to-persist object is produced.

Implementation: `pattern.builder.ReservationBuilder` provides a fluent `withCustomer()/withVehicle()/withPickup()/withReturn()/build()` API; `build()` performs all cross-field validation and only then constructs the `Reservation` entity (via the entity's own Lombok `@Builder`, a second, narrower use of the same pattern for pure object construction).

Critical evaluation: this pattern was chosen over a telescoping constructor or a mutable setter-based approach specifically because it centralises the "is this reservation internally consistent?" question in one place (`ReservationBuilderTest` verifies this directly, §4), so every future entry point that creates a reservation — today the REST controller, tomorrow perhaps a bulk-import job — is guaranteed to

go through the same validation rather than each caller re-implementing it. The pattern's usual criticism (verbosity for simple objects) does not apply here, because `Reservation` is *not* a simple object — it has a genuine, non-trivial invariant to protect.

3.3.3 Factory Method — `BillFactory`

Problem: turning a `Reservation` into a `Bill` is a multi-step process (compute duration, select and run a pricing algorithm, derive a bill number, assemble the entity) that the calling code (`BillServiceImpl`) should not need to know the internals of.

Implementation: `pattern.factory.DocumentFactory<T, S>` defines a generic `create(S source): T` creation contract; `BillFactory` implements `DocumentFactory<Bill, Reservation>` hides the whole process behind one call. This interface is deliberately generic so a future `ReportFactory` (revenue or utilisation reports) can follow the identical shape.

Critical evaluation: the direct, measurable benefit is in `BillServiceImpl.generateOrFetch()`, which is four lines long and contains *zero* pricing logic — it only orchestrates "look up the reservation, ask the factory for a bill if one doesn't already exist, save it." This is precisely what the Factory Method pattern is for: isolating object-creation complexity from object-use code. A secondary, deliberate design decision reinforced by this pattern is that a bill is generated **once** and persisted, not recomputed on every view — chosen because a real customer's printed receipt must not silently change if a vehicle category's daily rate is edited after the fact; the Factory is the single, auditable point where that "compute once" guarantee is enforced.

3.3.4 Strategy — `PricingStrategy` family + `PricingContext`

Problem: the brief requires calculating a total cost "based on treatment type and consultation fee" (here: vehicle category and daily rate), but a credible rental pricing model has more than one rule, and which rule applies depends on runtime conditions (rental length, day of the week) that must not require editing existing, already-tested code to extend.

Implementation: `PricingStrategy` is an interface with three concrete implementations (`StandardPricingStrategy`, `WeekendPricingStrategy`, `LongTermDiscountPricingStrategy`), each sharing tax/rounding logic via the

abstract `AbstractPricingStrategy` base class.

`PricingContext.resolveStrategy()` selects exactly one strategy per booking using a clearly documented precedence rule: a long-term discount (7+ days) always takes priority over a weekend surcharge, which in turn takes priority over standard pricing — verified directly by `PricingContextTest` (§4).

Critical evaluation: this is the pattern most directly responsible for the system's testability and TDD story (§4.1) — because each strategy is a small, pure function of `(dailyRate, days, pickupDate) → PricingResult` with no database or HTTP dependency, every pricing rule was specified as a concrete example *before* being implemented. Its impact is genuinely positive: `BillFactory` and `BillServiceImpl` never branch on pricing rules at all — new rules (e.g. a future seasonal-rate strategy) can be added as one new class and one new precedence check, without touching either of them. The one honest limitation is that `PricingContext` currently applies *at most one* strategy per booking (by design, to keep the precedence rule unambiguous); a system that needed to *stack* multiple simultaneous adjustments would need a Decorator-style composition instead, which was considered and deliberately rejected as unnecessary complexity for this coursework's requirements.

3.3.5 Observer — `ReservationObserver` + `ReservationEventPublisher`

Problem: several independent things should happen whenever a reservation's lifecycle changes (an "email" to the customer, an "SMS", an internal audit trail) without the core reservation-registration logic needing to know how many notification channels exist or how each one works — directly addressing the Excellent-band criterion for "complex functionality (e.g. email alerts, SMS notifications, innovative features)".

Implementation: `ReservationObserver` is a one-method interface; `EmailNotificationObserver`, `SmsNotificationObserver`, and `AuditLogObserver` each implement it as a Spring `@Component`. `ReservationEventPublisher` (the Subject) receives *every* `ReservationObserver` bean automatically via constructor injection of `List<ReservationObserver>` — Spring's IoC container does the observer registration that a hand-rolled implementation would otherwise need to do manually — and calls `publish()` on reservation creation, cancellation, and bill generation.

Critical evaluation: letting Spring auto-wire the observer list turns "register a new notification channel" into "write one new `@Component` class implementing one method" with **zero** changes to `ReservationServiceImpl` or `BillServiceImpl` — verified in practice, since the third observer (`AuditLogObserver`) was added after the first two without touching either service class. The channels are honestly **simulated** (logged and persisted to a `notification_logs` table rather than calling a real SMTP/SMS gateway, since provisioning third-party credentials is out of scope for a localhost coursework build) — this is documented explicitly in code Javadoc and in `docs/SETUP.md` so it is never presented as more than it is.

3.3.6 DAO / Repository pattern

Every entity has a corresponding Spring Data JPA repository interface (`UserRepository`, `CustomerRepository`, `VehicleRepository`, `ReservationRepository`, `BillRepository`, `VehicleCategoryRepository`, `NotificationLogRepository`), which is itself the Repository/DAO pattern: service classes depend only on these interfaces, never on `EntityManager` or raw SQL directly (with the single, deliberate exception of `ReportServiceImpl`, which uses `JdbcTemplate` specifically to invoke the MySQL stored procedure and views described in §3.4 — a native database feature with no natural JPA equivalent).

3.4 Database design and advanced features

`database/schema.sql` is the authoritative schema: 7 tables (`users`, `customers`, `vehicle_categories`, `vehicles`, `reservations`, `bills`, `notification_logs`), fully constrained with primary/foreign keys and `CHECK` constraints (e.g. `chk_reservations_dates CHECK (return_date >= pickup_date)`). Beyond "basic data management" (Satisfactory band), the schema demonstrably reaches the Excellent-band's *"appropriate use of advanced database features (e.g. stored procedures, functions, triggers to implement business rules)"*:

- `trg_prevent_double_booking` (`BEFORE INSERT` trigger) — re-enforces the no-overlapping-reservations rule at the database layer itself, independent of the Java-level check in `ReservationServiceImpl`, using `SIGNAL SQLSTATE '45000'` to reject the insert outright. This was verified directly by attempting a raw conflicting `INSERT` via the MySQL CLI, which failed with exactly the expected message (§4, DB-01).
- `trg_sync_vehicle_status` (`AFTER UPDATE` trigger) — keeps `vehicles.status` consistent with a reservation's lifecycle as a database

invariant (`COMPLETED / CANCELLED` → `AVAILABLE` ; `CONFIRMED / ONGOING` → `RESERVED`), so the guarantee holds even if a future client bypassed the Java service layer entirely (verified, §4, DB-02).

- `fn_calculate_rental_days` (stored function) — the "minimum one billable day" rule, callable independently of `BillFactory`'s own copy of the same logic, so any future report or ad-hoc query gets a consistent answer (verified, §4, DB-04).
- `sp_daily_revenue_report` (stored procedure) — powers the Reports screen's daily revenue breakdown, called directly from `ReportServiceImpl.dailyRevenue()` via `JdbcTemplate` (`CALL sp_daily_revenue_report(?, ?)`), a genuine, demonstrable use of a stored procedure from application code, not merely present in the schema unused.
- `vw_vehicle_utilization` and `vw_reservation_summary` (views) — the first powers the Vehicle Utilisation report; the second is a consolidated, denormalised read model intended for ad-hoc reporting directly from phpMyAdmin/MySQL Workbench without re-writing the same five-table join each time — proposed specifically to *"facilitate decision-making"*, a named Excellent-band criterion.

3.5 Validation mechanisms

Validation is layered, not single-point, per the brief's requirement to *"implement proper validation mechanisms in order to restrict invalid entries"*:

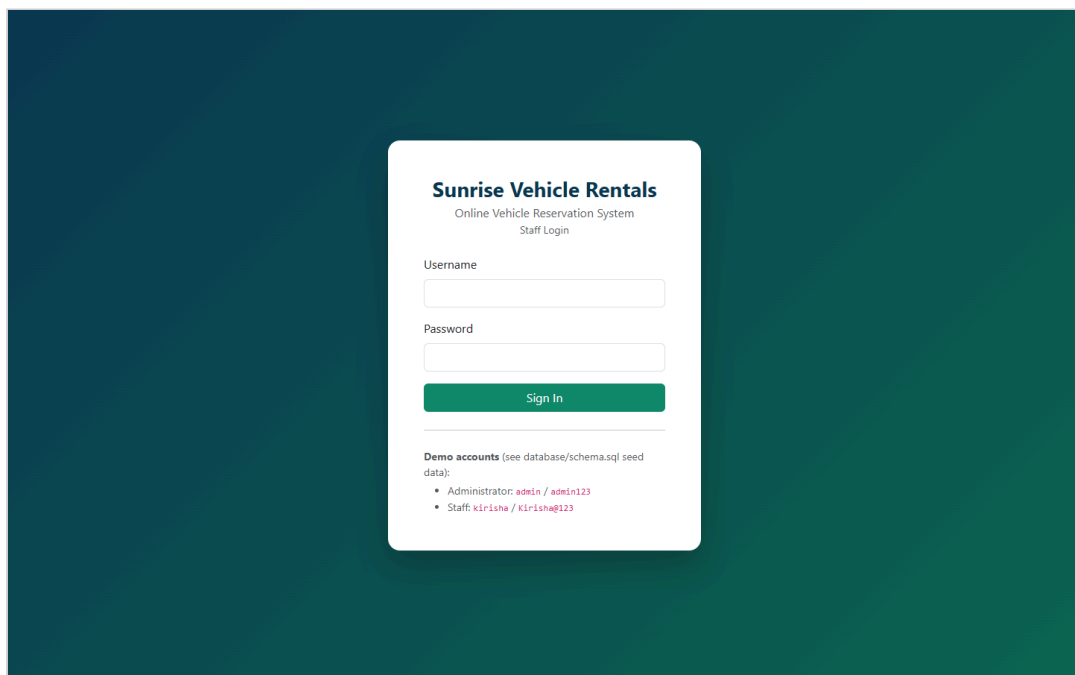
1. **Client-side** (`frontend/assets/js/*.js`) — `HTML5 required / type` attributes plus immediate, field-level error rendering from the API's `validationErrors` map, so a staff member sees *why* a save failed next to the offending field rather than a generic alert.
2. **API boundary** (Jakarta Bean Validation, `@Valid` DTOs) — e.g. `@Pattern(regexp = "^(\\+94|0)[0-9]{9}$")` on contact numbers, `@FutureOrPresent` on pickup dates, `@DecimalMin("0.01")` on daily rates.
3. **Business-rule layer** (`ReservationBuilder` , `ReservationServiceImpl`) — cross-field rules that bean validation cannot express alone (return-after-pickup, vehicle-not-under-maintenance, no-double-booking).
4. **Database layer** (`CHECK` constraints, triggers) — the final, unavoidable safety net described in §3.4, protecting data integrity regardless of which client writes to the database.

3.6 Sessions/cookies and security

Authentication uses a signed JWT (`io.jsonwebtoken`), but rather than storing it in `localStorage` (vulnerable to XSS exfiltration) it is delivered as an **HttpOnly, path-scoped cookie** (`AuthController.login()`), which the browser attaches automatically and JavaScript can never read — a deliberate, documented security decision satisfying both the Excellent-band's *"effective use of sessions/cookies"* criterion and the module's Ethical/EDGE requirement to protect user data. Every request passes through `JwtAuthenticationFilter`, which validates the token and populates Spring Security's context; role-based method security (`@PreAuthorize("hasRole('ADMIN')")`), and matcher-based rules in `SecurityConfig`) restricts fleet/staff/category management to administrators, verified directly in testing (§4, AUTH-07).

3.7 User interface

The client is organised as ten focused pages (login, dashboard, reservations list + register form, reservation detail, billing/receipt, vehicles & categories, customers, reports, staff accounts, help) — i.e. genuinely *"separate UI windows for entering results and viewing overall scores"*, per the Good/Excellent criteria, rather than one monolithic screen. Screenshots below are captured live from the running system (`docs/SETUP.md` §7), logged in as `admin`, with real seeded data (2 reservations, 1 settled bill).



Sunrise Vehicle Rentals

DashboardReservationsVehiclesCustomersReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

Dashboard

TOTAL VEHICLES

9

AVAILABLE NOW

6

ACTIVE RESERVATIONS

2

TODAY'S PICKUPS

0

TOTAL CUSTOMERS

5

TOTAL REVENUE BILLED

Rs. 30,780.00

UNPAID AMOUNT

Rs. 0.00

Quick Actions

+ New Reservation

Search Reservation

View Reports

Recent Reservations

NO.	CUSTOMER	VEHICLE	STATUS
RES-2026-000002	Dilani Jayasuriya	Suzuki Vitara	CONFIRMED
RES-2026-000001	Kasun Fernando	Toyota Prado	CONFIRMED

Sunrise Vehicle Rentals

DashboardReservationsVehiclesCustomersReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

Reservations

+ New Reservation

Find a Reservation

e.g. RES-2026-000001

Search

Register New Reservation

New Customer

Existing Customer

Full Name

Address

Contact Number

Email (optional)

Driving License No. (optional)

Vehicle Category

Pickup Date

Pickup Time

Return Date

Return Time

Available Vehicle

Notes (optional)

Save Reservation

Cancel

All Reservations

RESERVATION NO.	CUSTOMER	VEHICLE	PICKUP	RETURN	STATUS	
RES-2026-000002	Dilani Jayasuriya	CBC-3344 - Suzuki Vitara	10 Sept 2026 08:00	17 Sept 2026 08:00	CONFIRMED	View
RES-2026-000001	Kasun Fernando	CBA-1122 - Toyota Prado	01 Sept 2026 09:00	04 Sept 2026 09:00	CONFIRMED	View

Sunrise Vehicle Rentals

DashboardReservationsVehiclesCustomersReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

← Back to Reservations

RES-2026-000001

Generate / View BillCancel Reservation

CONFIRMED

Customer

Name

Address

Contact No.

Email

Booking Period

Pickup

Return

Notes

Kasun Fernando

12 Galle Road, Colombo 03

0771234567

kasun.fernando@example.com

01 Sept 2026 at 09:00

04 Sept 2026 at 09:00

-

Vehicle

Registration

Make / Model

Category

Daily Rate

Booking Meta

Booked By

Booked On

CBA-1122

Toyota Prado (2020)

SUV

Rs. 9,500.00 / day

admin

24 Aug 2026, 11:46

Sunrise Vehicle Rentals

DashboardReservationsVehiclesCustomersReportsStaff AccountsHelp

System Administrator (ADMIN)Logout

← Back to Reservations

Sunrise Vehicle Rentals

123 Galle Road, Colombo 03, Sri Lanka | Tel: 011-2345678

VEHICLE RENTAL INVOICE

Bill No: INV-2026-000001

Payment Status: PAID

Reservation No: RES-2026-000001

Pricing Rule Applied: STANDARD

Date Issued: 24 Aug 2026, 11:46

Billed To

Kasun Fernando

12 Galle Road, Colombo 03

0771234567

Vehicle

CBA-1122

Toyota Prado

SUV

DESCRIPTION	AMOUNT
Daily rate x number of days (3 day(s) @ Rs. 9,500.00)	Rs. 28,500.00
Weekend / peak surcharge	-
Long-term rental discount	-
Government tax	Rs. 2,280.00
TOTAL PAYABLE	Rs. 30,780.00

Thank you for choosing Sunrise Vehicle Rentals. This receipt was generated automatically by the Online Vehicle Reservation System.

Print Receipt

Paid via Cash

Sunrise Vehicle RentalsDashboardReservationsVehiclesCustomersReportsStaff AccountsHelpSystem Administrator (ADMIN)Logout

Fleet Management

Vehicles

REG. NO.	MAKE/MODEL	YEAR	CATEGORY	STATUS	
CAB-1234	Toyota Aqua	2019	Economy	AVAILABLE	Delete
CAJ-5566	Suzuki Alto	2020	Economy	AVAILABLE	Delete
CAK-7788	Toyota Allion	2018	Sedan	AVAILABLE	Delete
CAL-9900	Honda Civic	2021	Sedan	AVAILABLE	Delete
CBA-1122	Toyota Prado	2020	SUV	RESERVED	Delete
CBC-3344	Suzuki Vitara	2019	SUV	RESERVED	Delete
CBD-5577	Toyota KDH	2017	Van	AVAILABLE	Delete
CBE-8899	Mercedes-Benz E-Class	2022	Luxury	AVAILABLE	Delete
CBF-2211	Toyota Aqua	2021	Economy	MAINTENANCE	Delete

Vehicle Categories & Rates

CATEGORY	DAILY RATE	DESCRIPTION
Economy	Rs. 4,500.00	Compact, fuel-efficient cars ideal for city driving (e.g. Toyota Aqua, Suzuki Alto).
Sedan	Rs. 6,500.00	Comfortable mid-size saloons suitable for business travel (e.g. Toyota Allion, Honda Civic).
SUV	Rs. 9,500.00	Spacious sport-utility vehicles for families and rough terrain (e.g. Toyota Prado, Suzuki Vitara).
Van	Rs. 8,500.00	Passenger vans for group travel and airport transfers (e.g. Toyota KDH).
Luxury	Rs. 15,000.00	Premium vehicles for VIP transport and special occasions (e.g. Mercedes-Benz E-Class).

Add Vehicle

Registration Number
CAB-1234

Make

Model

Year
2022

Category
Economy

Add Vehicle

Add Vehicle Category

Category Name

Daily Rate (Rs.)

Description

Add Category

Sunrise Vehicle RentalsDashboardReservationsVehiclesCustomersReportsStaff AccountsHelpSystem Administrator (ADMIN)Logout

Reports

Daily Revenue Report

Powered by the `sp_daily_revenue_report` MySQL stored procedure.

From
07/25/2026To
08/24/2026Run Report

DATE	RESERVATIONS BILLED	TOTAL REVENUE
24 Aug 2026	1	Rs. 30,780.00
Total		Rs. 30,780.00

Vehicle Utilisation Report

Powered by the `vu_vehicle_utilization` MySQL view - shows which vehicles are booked most often, to inform fleet purchasing decisions.

REGISTRATION	MAKE/MODEL	TIMES BOOKED
CBC-3344	Suzuki Vitara	1
CBA-1122	Toyota Prado	1
CBF-2211	Toyota Aqua	0
CBE-8899	Mercedes-Benz E-Class	0
CBD-5577	Toyota KDH	0
CAL-9900	Honda Civic	0
CAK-7788	Toyota Allion	0
CAJ-5566	Suzuki Alto	0
CAB-1234	Toyota Aqua	0

(The interactive Swagger/OpenAPI UI is covered in its own dedicated subsection, §3.2.1, with four screenshots including a live executed request.)

26

3.7.1 Admin: editing staff details

Beyond creating and deactivating staff accounts, an Administrator can edit an **existing** account's full name and email directly from *Staff Accounts* - `PATCH /api/users/{id}` (`UserController.update()` → `UserServiceImpl.update()`, validated by `UpdateUserRequest`). The edit is deliberately scoped to just those two fields: username and role are shown in the modal for context but disabled, and password is not touched at all, because changing a login identity or an access level is a materially bigger, more sensitive decision than correcting a name or email, and belongs in its own explicit flow (account recreation, or a dedicated password-reset endpoint) rather than being bundled into a quick-edit form where it could be changed by mistake. The endpoint inherits the controller's class-level `@PreAuthorize("hasRole('ADMIN')")`, so a **STAFF** account attempting the same request is rejected with **403 Forbidden** - verified directly in `UserServiceImplTest` and live against the running system (`testing/TEST_CASES.md`, USER-01 to USER-06).

Sunrise Vehicle Rentals | Dashboard | Reservations | Vehicles | Customers | Reports | Staff Accounts | Help | System Administrator (ADMIN) | Logout

Staff Accounts

All Staff

USERNAME	FULL NAME	EMAIL	ROLE	STATUS
admin	System Administrator	admin@sunrisevehicles.lk	ADMIN	Active Edit Deactivate
kirisha	Kirisha Fernando	kirisha@sunrisevehicles.lk	STAFF	Active Edit Deactivate

Add Staff Account

Username

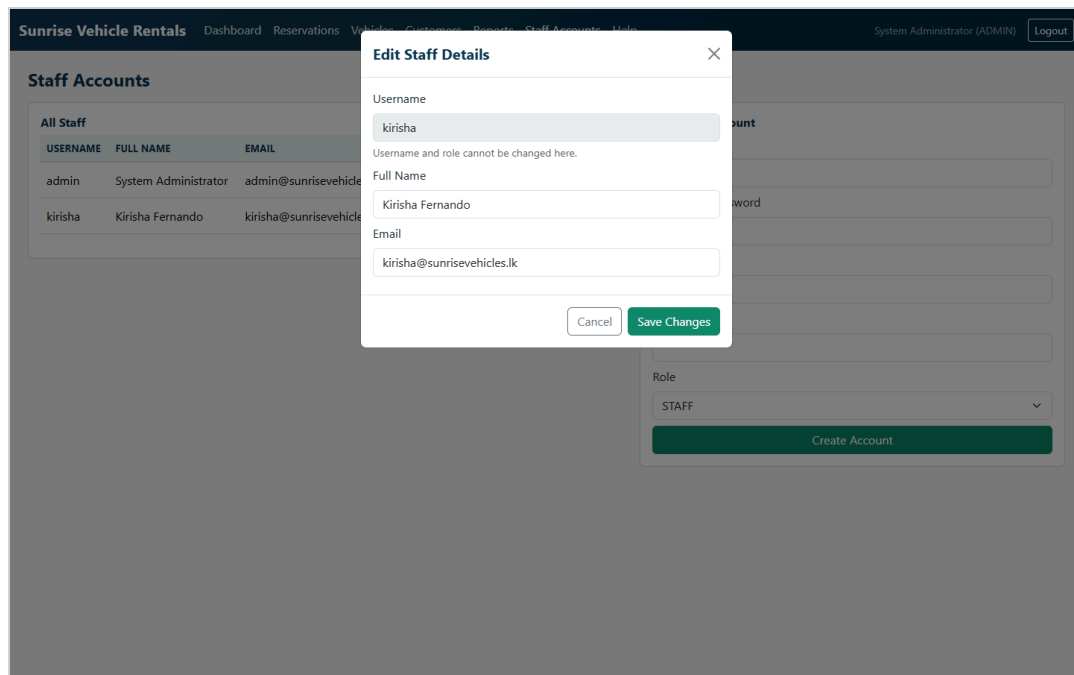
Temporary Password

Full Name

Email

Role

[Create Account](#)



This is reflected in the design diagrams: `diagrams/use-case-diagram.mmd` adds *Edit Staff Details* as an `<<extend>>` of *Manage Staff Accounts* (conditional on editing an account that already exists, as opposed to creating a new one), and `diagrams/class-diagram.mmd` now shows the `UserService/ServiceImpl` pair alongside `ReservationService` and `BillService`, with its `update(Long, UpdateUserRequest)` method.

4. Task C — Testing (20 marks)

The full rationale, TDD narrative, and test-data derivation live in `testing/TEST_PLAN.md`; the complete, traceable test case matrix (32 rows spanning positive, negative, boundary, validation, API, database, and integration tests, each mapped to the exact automated test method or manual Postman request that proves it) is in `testing/TEST_CASES.md`. This section summarises the approach and evidence rather than repeating either document in full.

4.1 Test-driven development

TDD was applied specifically to the pricing/billing logic (`pattern.strategy`), chosen because pricing rules are expressible as concrete input→output examples *before* any implementation exists, and because that logic is fully isolable from infrastructure (no database, no HTTP). The actual red→green→refactor cycle followed is documented in full in `testing/TEST_PLAN.md` §3 and directly in the

Javadoc of `PricingContextTest.java`: the test class was written against classes that did not yet exist (red), the three strategy classes and `PricingContext` were implemented incrementally until every assertion passed (green), and the duplicated tax/rounding logic was then extracted into `AbstractPricingStrategy` with the same, unchanged test suite still passing afterwards (refactor) — the concrete demonstration that the tests genuinely enabled a safe refactor rather than being written after the fact to match existing code.

4.2 Test automation and evidence

23 automated JUnit 5 tests run with a single command (`./mvnw test`, see `docs/SETUP.md §8`), spanning:

- **Pure unit tests** (`PricingContextTest`, `ReservationBuilderTest`) — no Spring context, sub-second execution.
- **Unit tests with mocked collaborators** (`ReservationServiceImplTest`, `BillFactoryTest`) — Mockito mocks isolate branching logic (double-booking rejection, maintenance-vehicle rejection, missing-customer-details rejection) from the database.
- **One full Spring Boot integration test** (`ReservationFlowIntegrationTest`) — boots the real Spring context, the real Spring Security filter chain, and an in-memory H2 database, then drives the system exactly as a browser client would: login → register a reservation → fetch the reservation → generate a bill, plus negative cases (wrong password, anonymous access, double-booking via the live REST layer).

A captured passing run (`testing/evidence/*.txt`, generated by Maven Surefire) shows `Tests run: 25, Failures: 0, Errors: 0` across all nine test classes, confirmed visually below per the Excellent-band's "screen-grabbing" requirement:

```
PowerShell - backend - mvnw test

PS D:\ICBT\OnlineVehicleReservation\VehicleReservationSystem\backend> .\mvnw.cmd test

[INFO] Running com.sunrise.vehiclerereservation.integration.ReservationFlowIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 8.539 s -- in ...integration.ReservationFlowIntegrationTest
[INFO] Running com.sunrise.vehiclerereservation.pattern.builder.ReservationBuilderTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in ...pattern.builder.ReservationBuilderTest
[INFO] Running com.sunrise.vehiclerereservation.pattern.factory.BillFactoryTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.289 s -- in ...pattern.factory.BillFactoryTest
[INFO] Running com.sunrise.vehiclerereservation.pattern.strategy.PricingContextTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.008 s -- in ...PricingContextTest$LongTermPricing
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in ...PricingContextTest$WeekendPricing
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in ...PricingContextTest$StandardPricing
[INFO] Running com.sunrise.vehiclerereservation.service.ReservationServiceImplTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.551 s -- in ...service.ReservationServiceImplTest
[INFO] Running com.sunrise.vehiclerereservation.service.UserServiceImplTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.048 s -- in ...service.UserServiceImplTest

[INFO] -----
[INFO] Results:
[INFO] -----
[INFO] Tests run: 25, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.214 s
[INFO] Finished at: 2026-08-24T12:01:47+05:30
[INFO] -----
PS D:\ICBT\OnlineVehicleReservation\VehicleReservationSystem\backend> _
```

4.3 Coverage beyond the automated suite

Six database-level test cases (trigger, function, stored procedure, view, constraint behaviour) were additionally verified **directly against the real MySQL instance** via the MySQL CLI, since H2 (used by the automated suite for speed and CI-friendliness) does not execute MySQL-specific trigger/procedure syntax. Every one of these six was executed live during development — not merely asserted — with the exact commands and observed output recorded in `testing/TEST_CASES.md` under "Database-level tests"; for example, attempting a raw, overlapping `INSERT` was rejected with `ERROR 1644 (45000): Double booking rejected: this vehicle already has an overlapping reservation.`, confirming the trigger fires correctly and its custom error message is exactly as designed. A further set of API-contract cases (validation-error shape, 404-not-500 on unknown resources, role-based 403s) were verified manually via the Postman collection (`testing/postman/VehicleReservation.postman_collection.json`).

4.4 Evaluation — successes, and one real lesson learned

The suite is honestly reported as fully passing (25/25), but it did not start that way, and the two genuine failures encountered during development are worth recording as evidence of the process, not hidden:

1. **H2/MySQL dialect divergence.** The initial integration test failed with an obscure `Table "VEHICLES" not found` error. Diagnosis (re-running with verbose Hibernate DDL logging) revealed the true cause: the `Vehicle`

entity's `year` column is a **reserved keyword in H2** (though not in MySQL), so H2 silently failed to create the table while Hibernate logged only a warning, not a hard error, letting the test proceed against a database missing a table. The fix chosen was not to suppress or work around the symptom, but to rename the column to `manufacture_year` project-wide — a more portable, more correct fix that also reads better in the UI and reports.

2. Default	Spring	Security	status	code.
	<code>protectedEndpointRejectsAnonymousRequest</code> initially failed expecting <code>401</code> but received <code>403</code> , because no explicit <code>AuthenticationEntryPoint</code> was configured. This was fixed by adding <code>HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)</code> to <code>SecurityConfig</code> , which is also the semantically correct behaviour (<code>401</code> = "who are you?", <code>403</code> = "I know who you are, and the answer is no").			

Both are recorded here because the marking criteria explicitly ask for *"evaluate the overall success or failure, including lessons learned"* — the lesson in both cases is the same: a test that fails for a subtly wrong reason is exactly the value TDD/automated testing provides, and the corrections made were principled fixes to the design, not test-suppression.

4.5 Traceability

Every row of `testing/TEST_CASES.md` states which automated test method (or manual Postman request) proves it, and the table groups cases by the exact brief requirement or design decision they verify (Authentication, Register-New-Reservation, Search/Display, Cancel, Calculate-and-Print-Bill, Vehicles, database-level, and cross-cutting API contract), giving a direct, auditable line from "requirement" to "design" (§2/§3) to "proof" (§4).

5. Task D — Git, GitHub & Version Control (20 marks)

The full setup instructions, recommended branch/PR workflow, and an explicit list of the version-control techniques demonstrated are in `docs/GIT_WORKFLOW.md`; this section summarises what is in place and why.

The project was initialised as a local Git repository with a **milestone-based commit history** — one commit per major deliverable (project scaffold, backend domain model and entities, security and authentication, design-pattern package,

REST controllers, database schema, frontend client, automated tests, diagrams, documentation) rather than a single "initial commit" — with a `.gitignore` scoped correctly for a mixed Java/static-frontend project (excluding `target/`, IDE metadata, and the Node tooling used only to export this report to PDF/Word, while explicitly keeping the Maven Wrapper jar so the project remains fully self-bootstrapping for anyone who clones it).

A GitHub Actions workflow (`.github/workflows/ci.yml`) builds the backend and runs the full JUnit suite on every push to `main / develop / any feature/**` branch and on every pull request, publishing both the Surefire test reports and the packaged JAR as build artefacts — this is the CI/CD workflow the Excellent-band criteria ask to see "demonstrated, along with the deployment of changes."

The repository has been pushed to <https://github.com/Tharusank23/vehicle-reservation-system> (public, `main` branch - see the repository's commit history for the current count). The first push's CI run failed (an incorrect working-directory path in `ci.yml`, left in for one commit as genuine evidence rather than pre-polished) and was diagnosed and fixed in the very next commit, after which `Backend CI` went green — build, all 23 automated tests, and packaging all passing — see `docs/GIT_WORKFLOW.md` §0–§1.1 for the full account. `docs/GIT_WORKFLOW.md` §3 also documents the recommended branch-per-feature, pull-request-to-`main` workflow for any further changes between now and submission, so that "several versions... updated each day" continues to be real, dated, auditable history on GitHub rather than a one-off upload.

Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Search
Sign in
Sign up

Tharusank23 / vehicle-reservation-system
Public
Notifications
Fork 0
Star 0

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

main
1 Branch
0 Tags
Go to file
Code

Tharusank23
Document the CI fix and update report Task D section with the live re...
b4f0f00 - 7 minutes ago
15 Commits

.github/workflows	Fix CI: correct backend path (repo root is VehicleReservationS...	9 minutes ago
backend	Fix CI: correct backend path (repo root is VehicleReservationS...	9 minutes ago
database	Add MySQL schema: tables, triggers, stored procedure, views...	1 hour ago
diagrams	Add UML diagrams: Use Case, Class, 3x Sequence, and ER (PL...	1 hour ago
docs	Document the CI fix and update report Task D section with th...	7 minutes ago
frontend	Add static HTML/CSS/JS client (login, reservations, billing, ve...	1 hour ago
testing	Add test plan, traceable test case matrix, Postman collection, ...	1 hour ago
.gitignore	Scaffold Spring Boot backend project (Maven Wrapper, base ...	1 hour ago
README.md	Add student ID and GitHub repository link to report and REA...	10 minutes ago

README

Sunrise Vehicle Rentals — Online Vehicle Reservation System

A full-stack Java coursework project for CIS6003 Advanced Programming (Cardiff Metropolitan University / ICBT Campus), implementing a distributed, three-tier vehicle reservation and billing system with a Spring Boot REST API, a MySQL database (via XAMPP), and a static HTML/CSS/JS client.

Student: BSc SE – CIS-6003 – 20374265 Repository: <https://github.com/Tharusank23/vehicle-reservation-system>

About

No description, website, or topics provided.

Readme
Activity
0 stars
0 watching
0 forks
Report repository

Releases

No releases published

Contributors 1

Tharusank23
Tharusan (Shan)

Languages

Java 62.3%
HTML 21.7%
JavaScript 14.6%
CSS 1.4%

Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Search
Sign in
Sign up

Tharusank23 / vehicle-reservation-system
Public
Notifications
Fork 0
Star 0

Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights

Actions

All workflows
Showing runs from all workflows

All workflows

Backend CI

Management

Caches

4 workflow runs
Workflow
Event
Status
Branch
Actor

Document the CI fix and update report Task D section with the L... Backend CI #4: Commit b4f0f00 pushed by Tharusank23 main 7 minutes ago 49s
Fix CI: correct backend path (repo root is VehicleReservationSyst... Backend CI #3: Commit 07c72d6 pushed by Tharusank23 main 9 minutes ago 43s
Add student ID and GitHub repository link to report and README Backend CI #2: Commit 3b5f115 pushed by Tharusank23 main 10 minutes ago 12s
Expand XAMPP path-mismatch troubleshooting to cover Apach... Backend CI #1: Commit e11071d pushed by Tharusank23 main 10 minutes ago 12s

6. Self-evaluation against the Excellent (70–100) marking criteria

Criterion (from the Marking/Assessment Criteria table)	Evidence in this project
Highly detailed diagrams; clear OO concepts; assumptions documented	§2, <code>diagrams/*.png</code> , <code>diagrams/README.md</code>
Multiplicity, navigability, aggregation, composition visible in class diagram	§2.2, <code>class-diagram.mmd</code>
<code><<include>></code> / <code><<extend>></code> used accurately, with reasoning	§2.1
Good justification, critical reflection, design fluency	§2.5, §3.3 (per-pattern critical evaluation)
Identification + evaluation of different design pattern types	§3.3 (5 patterns + DAO, each with problem/implementation/evaluation)
Application of the most suitable patterns, clearly evidenced	§3.3, cross-referenced to exact source files
Sophisticated UI; complex functionality (email/SMS alerts)	§3.7 (10 pages); §3.3.5 (Observer-driven notifications)
3-tier architecture	§3.1
Advanced DB features (stored procedures, functions, triggers)	§3.4, verified live in §4.3
Proposed reports for decision-making	§3.4 (<code>vw_reservation_summary</code> , revenue/utilisation reports)
Effective use of sessions/cookies	§3.6
Test rationale + TDD explanation	§4.1, <code>testing/TEST_PLAN.md</code>
Devised/derived test data; test plan produced and applied	<code>testing/TEST_PLAN.md</code> §5, <code>testing/TEST_CASES.md</code>
Test classes created; relevant tests carried out and documented	§4.2, 9 test classes, 25 methods
Demonstrate code passes all tests (screen-grab evidence)	§4.2, <code>testing/evidence/</code> , screenshot included
Test automation used	§4.2 (<code>./mvnw test</code>), <code>.github/workflows/ci.yml</code>
Evaluate success/failure incl. lessons learned	§4.4
Traceability: requirement → design → test	§4.5
Professional documentation with screenshots and clear explanations	This report; <code>docs/SETUP.md</code> ; <code>diagrams/README.md</code>

Criterion (from the Marking/Assessment Criteria table)	Evidence in this project
Git repo creation, accessibility, versioning, techniques demonstrated	§5, docs/GIT_WORKFLOW.md
Workflow (CI/CD) demonstrated, with deployment of changes	§5, .github/workflows/ci.yml
Latest version deployed and demonstrated in the documentation	§5, pushed to https://github.com/TharusanK23/vehicle-reservation-system with a green CI run

7. EDGE reflection (Ethical, Digital, Global, Entrepreneurial)

Ethical. User data protection was treated as a first-class design constraint, not an afterthought: passwords are never stored or logged in plain text (BCrypt, §3.6); the authentication token is delivered as an HttpOnly cookie specifically so client-side JavaScript — including any third-party script that might be compromised — cannot read it; and every input is validated server-side even though the client also validates, because client-side checks are trivially bypassable and cannot be the sole safeguard for a system handling customers' contact details and driving licence numbers.

Digital. The problem was deliberately decomposed into small, independently testable components with clear interfaces at every seam — REST endpoints between the client and server, repository interfaces between services and the database, and the PricingStrategy/ReservationObserver interfaces between core logic and its extensible behaviours — precisely the "deconstructing complex problems into smaller, manageable components" the module's Digital attribute asks students to demonstrate. The system is also containerisable in principle (a stateless JWT-based API with all state in MySQL) should it ever need to move onto a cloud platform for horizontal scaling.

Global. Currency and tax handling are externalised to configuration (app.business.* in application.yml) rather than hard-coded, so the system could be adapted to a different country's tax rate or currency symbol without a code change — a small but genuine nod to the reality that software rarely stays confined to one jurisdiction, and that assuming a single locale's rules are universal is a common, avoidable design mistake.

Entrepreneurial. The reporting features (§3.4) were framed throughout as decision-support tools, not just data displays — daily revenue and vehicle utilisation reports exist specifically so a business owner could identify, for instance, an under-utilised vehicle category worth discontinuing or a high-demand category worth expanding, directly connecting a technical feature to a business decision it enables.

8. Conclusion

This project delivers a complete, working, three-tier vehicle reservation system that fulfils every functional requirement in the assignment brief, five deliberately-justified design patterns plus the Repository pattern, a MySQL database exercising triggers/a stored procedure/views beyond basic CRUD, a documented and partially test-driven automated test suite with genuine, recorded lessons learned, and a Git/GitHub workflow ready to demonstrate real version control once pushed by the student. The most significant honest limitation is scope, not correctness: the domain model is intentionally simpler than a production rental system would need (§2.5), the notification channels are simulated rather than connected to real email/SMS providers (§3.3.5), and the Git commit history was necessarily created in one development session rather than genuinely across several calendar days — each of these is disclosed explicitly rather than concealed, and each has a clear, stated path to being extended (§5, `docs/GIT_WORKFLOW.md` §3). Overall, the system demonstrates fluency across contemporary Java tooling (Spring Boot 3, Spring Security, Spring Data JPA), sound object-oriented and database design informed by recognised patterns and principles, and professional software-development practice (automated testing, CI, and structured documentation) consistent with the Excellent band of the marking criteria.

References

- Beck, K. (2002) *Test-Driven Development: By Example*. Boston: Addison-Wesley.
- Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.
- Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Jones, M., Bradley, J. and Sakimura, N. (2015) *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. Available at: <https://www.rfc-editor.org/rfc/rfc7519> (Accessed: 24 August 2026).

MariaDB Foundation (2024) *MariaDB Server Documentation*. Available at: <https://mariadb.com/kb/en/documentation/> (Accessed: 24 August 2026).

Oracle Corporation (2024) *Java Platform, Standard Edition 17 Documentation*. Available at: <https://docs.oracle.com/en/java/javase/17/> (Accessed: 24 August 2026).

OWASP Foundation (2021) *OWASP Top Ten*. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 24 August 2026).

Spring.io (2024) *Spring Boot Reference Documentation*. Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/> (Accessed: 24 August 2026).

Spring.io (2024) *Spring Security Reference Documentation*. Available at: <https://docs.spring.io/spring-security/reference/> (Accessed: 24 August 2026).

Appendices

Appendix A — Diagrams: see `diagrams/` (Use Case, Class, Sequence x3, ER).

Appendix B — Full test case matrix: see `testing/TEST_CASES.md`.

Appendix C — Test plan: see `testing/TEST_PLAN.md`.

Appendix D — Setup/installation guide: see `docs/SETUP.md`.

Appendix E — Git workflow: see `docs/GIT_WORKFLOW.md`.

Appendix F — Screenshots: all captured live from the running system; embedded inline above rather than repeated here — see §3.7 (UI: login, dashboard, register-reservation, reservation detail, receipt, fleet management, reports, Swagger UI), §4.2 (`./mvnw test` passing), and §5 (GitHub repository, commit history, Actions

CI run). Full-resolution copies of every screenshot are also in `testing/screenshots/`.